

Q Design a circuit that generate Pulse on every rising edge of input

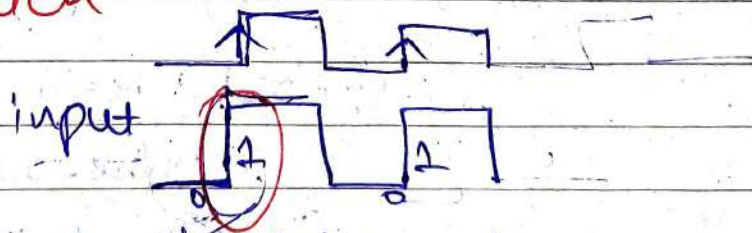
Intuition →

Is input synchronous to clock?

Should output pulse width
Do we need ~~would~~ be exactly 1 clock cycle?
a reset signal

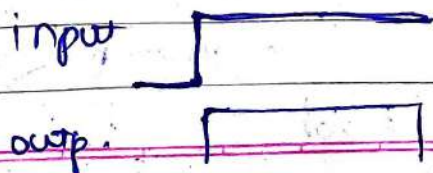
→ Before implementing, I will assume the input is synchronous to the clock, reset is active high, output should generate a single cycle pulse

Step ② Approach



to detect rising edge

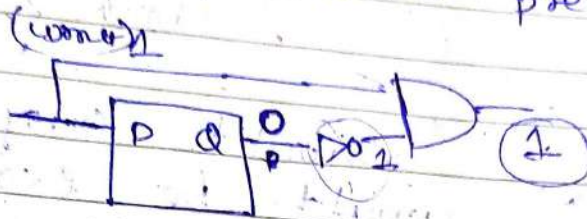
I should have previous value & current value of signal



$1 \oplus 1 = 0$

So, I'll store previous value of input using flip flop.

Then I'll compare current input value with previous value.



Step 3 Verilog coding

```
module edge_detector (
    input clk, rst, in,
    output reg pulse_out
);
```

Sequential logic is needed

```
    reg in_prev;
```

```
always @(posedge clk) begin
```

```
    if (rst) begin
```

```
        pulse_out <= 0;
```

```
        in_prev <= 0;
```

if not initialized it will result as (X unknown) end

```
    else begin
```

```
        pulse_out <= in & (~in_prev)
```

```
        in_prev <= in;
```

```
    end
```

```
endmodule
```

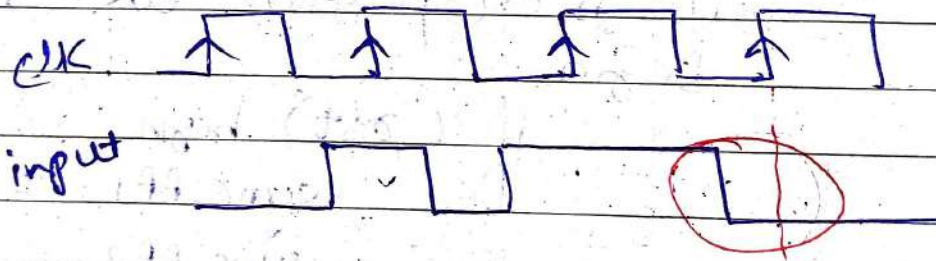
Step 4 verification thinking

I would verify functional behaviours
in corner cases

prev current output

0	0	0
0	1	1
1	0	0
1	1	0

follow up
one if input is asynchronous?



→ if I try to sample input near
clock edge I can observe
input changes frequently

this may violate setup/hold
of flip flop

↓
we get unpredictable output

approach $\rightarrow$ I would first use
2 flop synchronizer
to safely bring the
signal into clock domain.
then perform edge
detection

verilog code:

2 ff synchronizer

```
reg sync_ff1, sync_ff2;

always @(posedge clk or posedge rst)
begin
    if (rst) begin
        sync_ff1 <= 1'b0;
        sync_ff2 <= 1'b0;
    end
    else begin
        sync_ff1 <= async_in;
        sync_ff2 <= sync_ff1;
    end
end
```

why
edge detection
logic after
synchronization?

end

```
reg sync_ff2_prev;
```

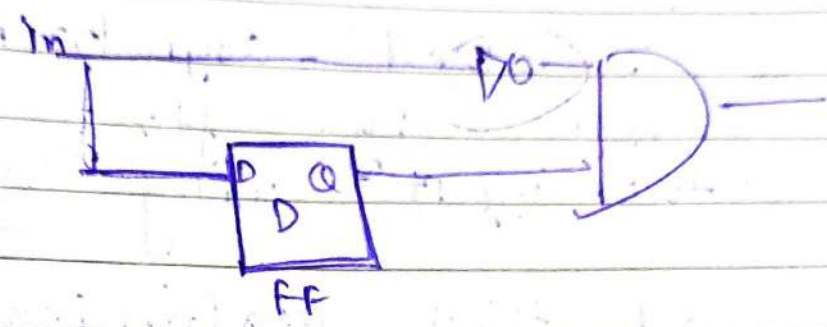
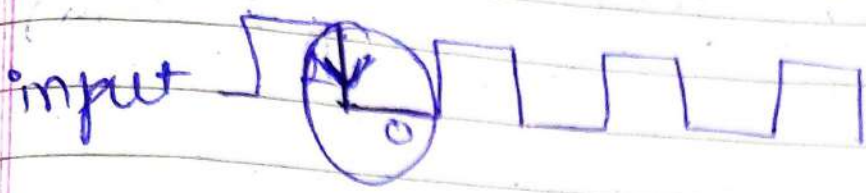
because metastable signals should
never feed combinational logic
in $\&$ (min)

Summary

I solved asynchronous edge detection
problem by first synchronizing
the input using 2 FF Synchronizer
and then performing synchronous
Edge detection

This avoids metastability propagation
and generate single cycle pulse.

if negative edge detection



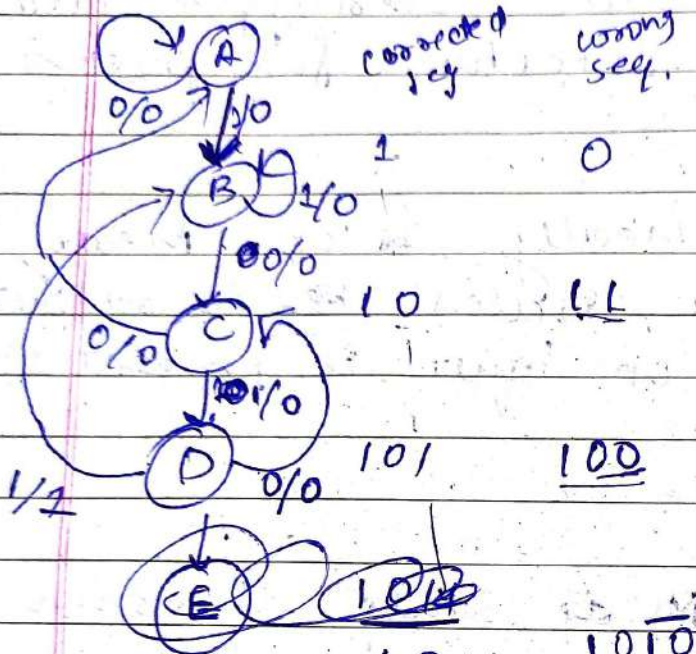
current input previous output

	0	0	0
→	0	1	1
	1	0	0
	1	1	0

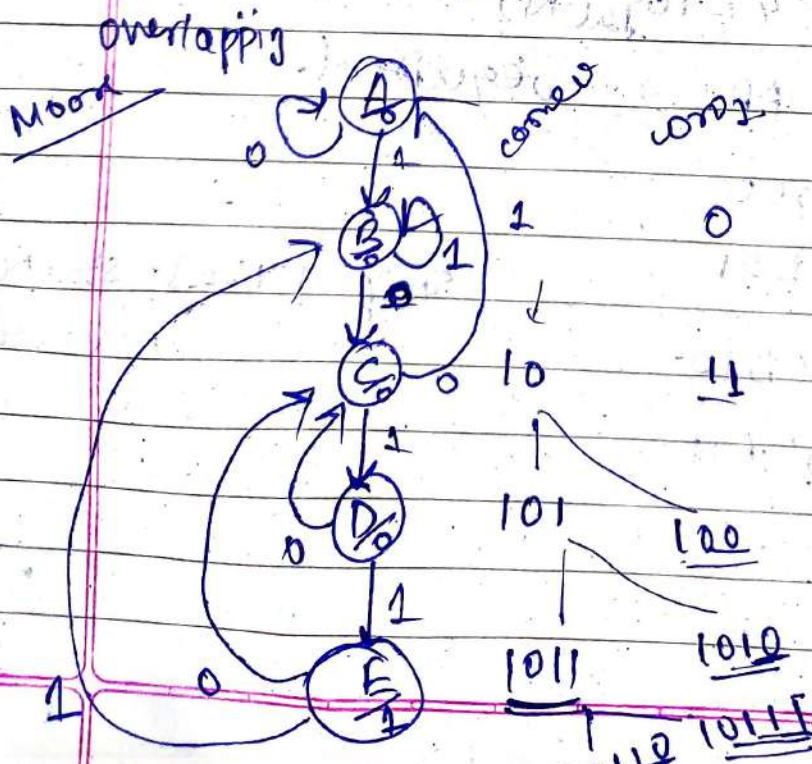
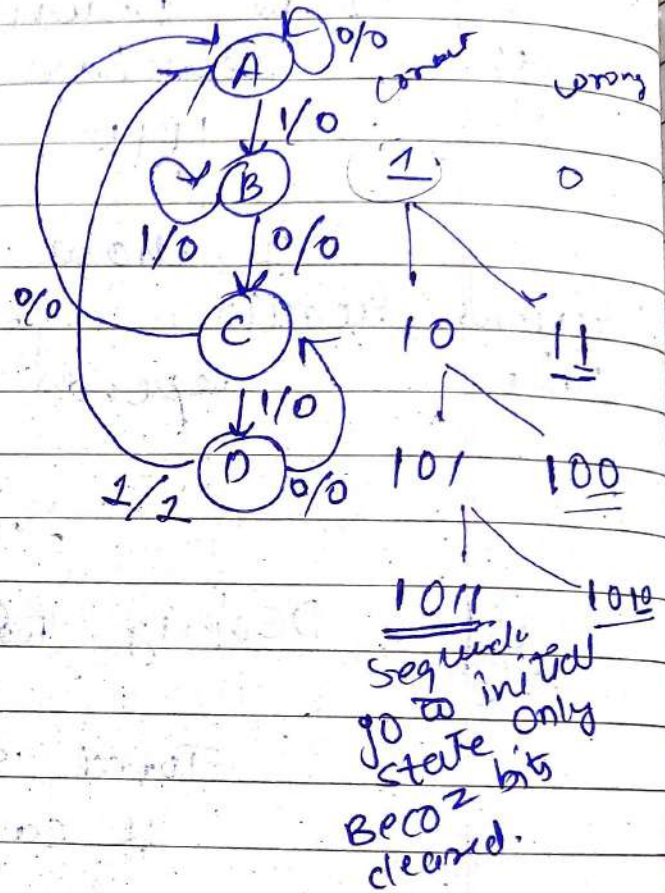
sequence detector 1011

overlapped mealy.

states = 4



Non overlapped mealy.



If Ask for overlapping
non overlapping sequence ??

Step 1

I'll assume an overlapping Mealy sequence detector for pattern 1011

I chose Mealy becoz Mealy responds fast, requires few states. As output depends on input & present state

Step 2

Designing ~~finite~~ state diagram

Total states = 4

I can use binary encoding style for state representation

$$4 = \log_2(N)$$

If 2 required.

$$A = 2'b00$$

$$B = 2'b01$$

$$C = 2'b10$$

$$D = 2'b11$$

eg [1:0] state, next state

Step 3 verilog code.

```
module seq-detector(
```

```
    input clk, rst, in  
    input  
    output Out  
);  
;
```

Step 4 let me verify

input stream

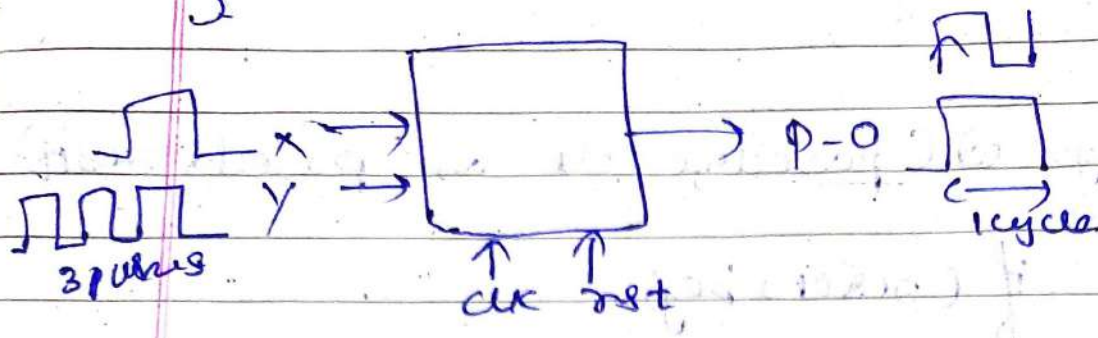
1 0 1 0 0 1 1

↓
1

↓
1

2 detections

8



let me summarize the requirement first

→ p-o signal is asserted for exactly 1 clock cycle only if 3 pulses occurred on signal y & pulse on x

Edge cases

- ① count < 3 p-o = 0 (No pulse)
- ② count > 3 No pulse
- ③ x pulse on same cycle as y pulse y counts toward next window
- ④ reset counter = 0
p-o = 0

my approach

always @ (posedge clk or posedge reset)

begin

if (reset) begin

y-count <= 0;

p-o <= 1'b0;

end

else begin

if (y-count == 3 && x[i])

p-o <= 1'b1;

else if (y-count < 3 && x[i])

p-o <= 1'b0;

correct detection

But

p-o => remain high

for more 1 clock cycle

Also

counter should reset

Spec:- exactly high for 1 cycle

→ so take default p-o <= 1'b0;

→ 1 AS7C interview evaluate

✓ corner cases

✓ sequential ordering

Not just logic

~~But~~

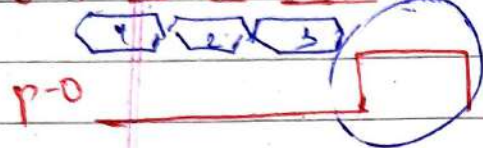
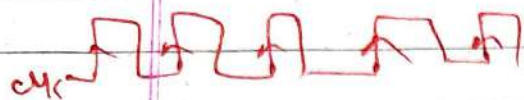
But when z simulated

if $(x-i)$ begin

if $(y-count \geq 3)$ begin

$p-o \leq 1'b1$;
 $y-count \leq 0$;

end



generated
after

complete 3 count.

Beoz we have Non
Blocking

~~failed
test~~

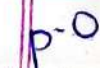
debug

assign $p-o = (y-count == 3$
&
 $(x-i)$

use
assign
"combinational
logic"

to immediately

assign value to Output signal.



8bit 8bit 8bit 8bit

9 7 5 4 2 1 8 6 = → second largest value only

~~8 bit data input~~
Stream of unsigned integers

①

↑ 9 OUT = 0 0

↑ 7 compare OUT ⇒ $\frac{9}{\text{max}}$ 0

↑ 5 compared. OUT = $\frac{9}{\text{max}}$ 7

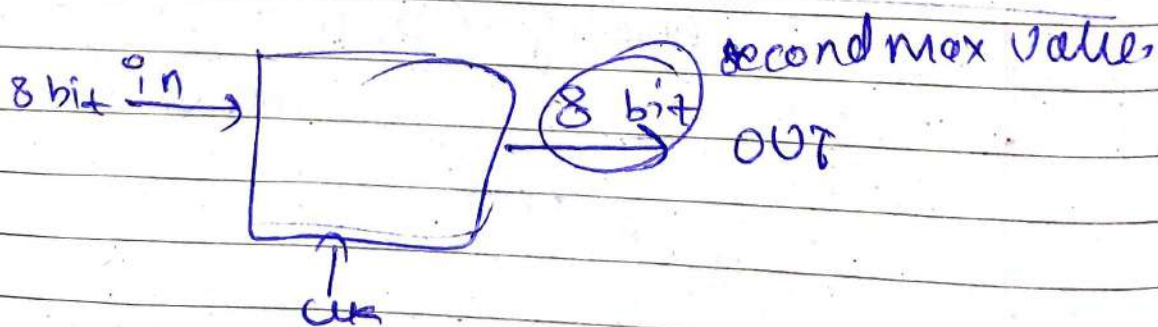
↑ 4 OUT = $\frac{9}{\text{max}}$ $\frac{7}{\text{max}}$

↑ 2 OUT = $\frac{9}{\text{max}}$ $\frac{7}{\text{max}}$ second MAX

↑ 1 OUT = $\frac{9}{\text{max}}$ $\frac{7}{\text{max}}$

↑ $\frac{8}{\text{max}}$ OUT = $\frac{9}{\text{max}}$ $\frac{8}{\text{max}}$ second MAX

OUT ⇒ 0 7 7 7 7 8



we need to continuously process 8 bit stream of input & keep the track of 1st largest

2nd Largest

Output only 2 max value every
clock edge,

```
reg [7:0] max1, max2;
```

always @C posedge clock ^{or negedge rst} begin

(2)

if (reset) begin

max1 <= 0;

max2 <= 0;

end

else begin

if (DZN >= max1) begin

~~max1 <= DZN;~~

~~max2 <= max1;~~

max2 <= max1;

max1 <= DZN;

end

else if (DZN >= max2) begin

max2 <= DZN;

end

end

end

assign DOUT = max2;

end module

always
+
coding

③ I would verify both normal & edge case scenario

Normal	Repeated
→ 9875	5555
→ 0555 0888	0555

④ overall I used 2 register continuously track the first Largest & second Largest value

Decade Counter

→ I need to design modulo-10 counter 0-9 then wrap back to 0

① problem to verification

Reset → synchronous. which means counter reset on active clock edge.

$10 < 2^4$ 4 bit flipflops

② approach

I'll use a 4 bit register.

On every clock edge, if reset is asserted then set counter to 0 otherwise increment count until count reaches to 9, wrap back to 0

③ coding

④ verification

verify illegal states 10, 11, 12, 13, 14, 15

Test case ① Normal counting

↑	0
↑	1
↑	2
↑	3

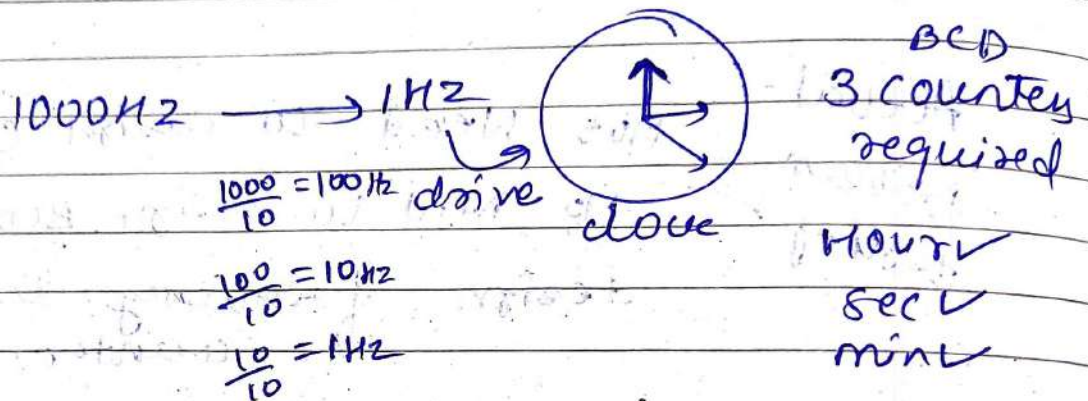
Test case ② Reset

↓ count = 7
reset = 1

↓ count = 0

Q generate 1 Hertz pulse from a 1000Hz clock

I need to use frequency divider using (decade) MOD-10 (BCD) counter



1000Hz $\rightarrow$ No. of cycles per second (frequency)

1Hz $\rightarrow$ 1 pulse every second

Intuition

If ~~second~~ counter connected directly to 1000Hz freq:

So 0
1
2
3
...
1000

$\rightarrow$ would count 1000 times per second which is wrong.

1000Hz
 $\downarrow$
frequency divider
 $\downarrow$
1Hz
 $\downarrow$
second counter

see.

0
1 ← increments (1) once every second
2
3
⋮

Actual
problem
testing

Not Need to design digital clock

No Need to design BCD counter
design of frequency divider using
counter.

BCD counter → counts 0 to 9
→ divides the freq
~~counter divide by~~ 10

BCD counter : 0 1 2 3 4 5 6 7 8 9 → 0

~~counter~~

count[2] count[1] count[0]

000

001

002

003

004

009

010

011

099

100

increment
only

when

count = 9

count[0] = 9

increment
only when
count[0]
reaches
to
9

2

3

4

5

6

7

8

9

00

01

02

03

04

05

06

07

08

09

10

11

12

13

14

15

999

Now

1Hz

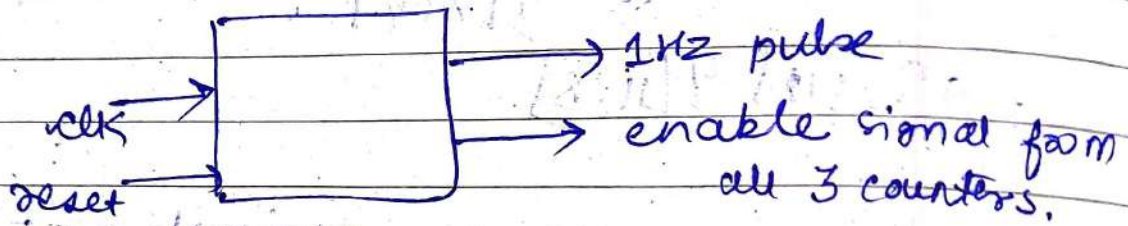
becoms.

1

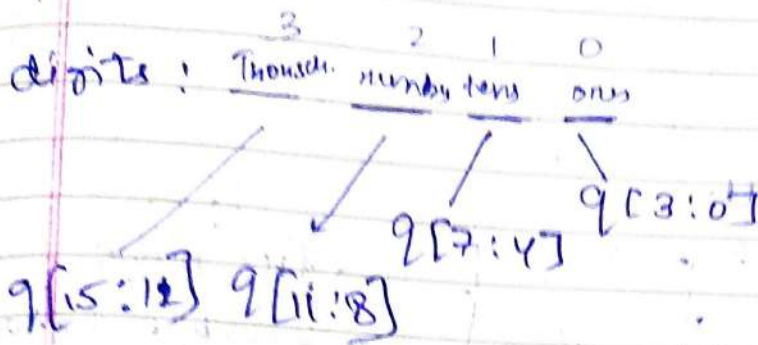
1s

1000000 1000000
1Hz exactly 1 clock cycle per sec.

Identify
input & output



Q Build 4 digit BCD counter



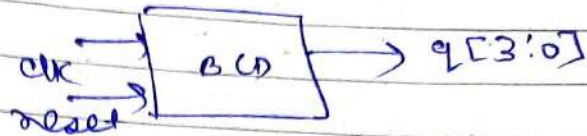
each ~~decimal~~ digit counts from 0 to 9 decimal value

count sequence

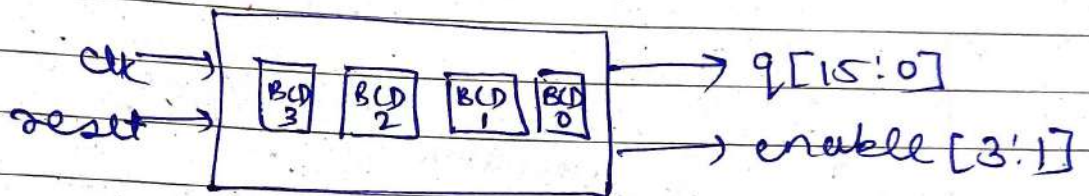
```

0 0 0 0
0 0 0 1
0 0 0 2
0 0 0 3
0 0 0 9
-----
0 0 1 0
      1 1
      1 2
      1 3
      1 3
-----
0 0 9 9
-----
0 1 0 0
      1 1
      1 1
-----
0 9 9 9
-----
1 0 0 0
-----
9 9 9 9
  ↓ ↓ ↓ ↓ wrap
  0 0 0 0
  
```

we may have or instantiate
1 digit BCD counter (decade MOD 10)



for Top level design.



✓ 4 cascaded decade counters

✓ instantiate this BCD counter in top module

My Mistake
in code

Error!
Q has aggregate value

output reg Q[3:0];

this becomes
meaning unpacked array



Q <= 4'd0;

Q = 4'd9;

Q + 4'd1;

operations applicable when
Q → vector

Not an array

change

output reg [3:0] Q; ✓ vector

Design Digital 12hr Clock

synchro.
Reset

higher priority than Enable signal

AM = 0
PM = 1 } signal (pm)

hh
mm
ss
BCD digits &
2 digits.

11:59:59 AM ^{rollover} 12:00:00 PM

reset the clock to 12:00 AM.

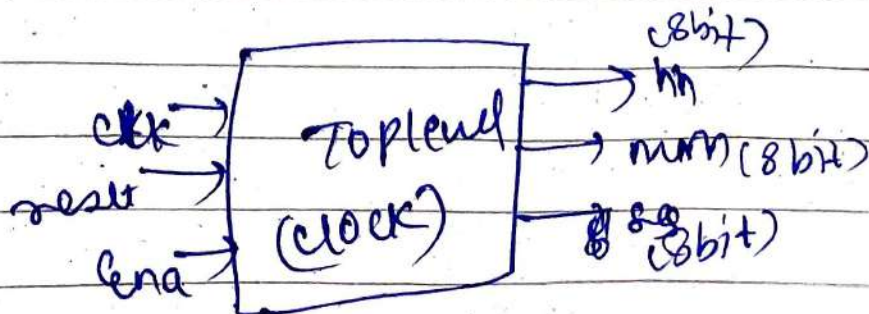
→ approach:

we need BCD counter

so that counts from 0 to 9 so that we can have time as

hh:mm:ss
0 to 9 0 to 9

q[3:0] BCD counter output



Not Full BCD counter.
 SS 0 to 59. mm 0 to 59. But modulo 6. BCD counts 5 9.

First I would solve second & minutes generation using counter logic & then instantiating this counter module.

once min, sec generated treating hours separately because the constraint in hours is

11:00 AM $\xrightarrow{\text{Rollover Logic}}$ 12:00 PM

12:00 PM $\xrightarrow{\text{Rollover Logic}}$ 01:00 PM

00 $\rightarrow$ 01 $\rightarrow$ 02 $\rightarrow$ 03 $\dots$ 09 $\rightarrow$ 10 $\rightarrow$ 11 $\rightarrow$ 12

[7:0] hh = $\frac{2,1,0}{h[7:4]} \frac{9,8,0}{h[3:0]}$

$\rightarrow$ coding

$\rightarrow$ Verification

Test 1: Reset 12:00:00 AM

Test 2: Second Rollover

12:00:59
 12:01:00

Test 3: Minute Rollover

12:59:59
 01:00:00

Test 5: Noon/Midnight

12:00:00 PM
 01:00:00 PM

Test 4: AM/PM toggle

11:59:59 AM
 12:00:00 PM

Triangular wave Generate

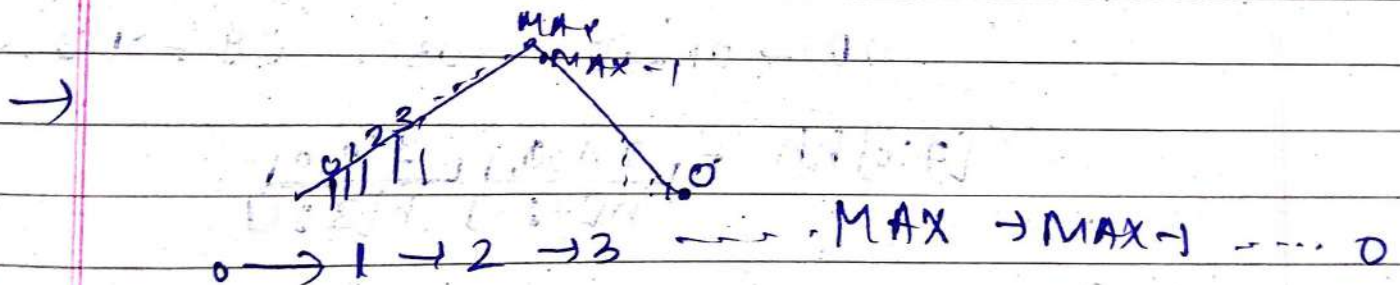
module trianglewave_gen # (parameter
WIDTH = 8)

(
input clk,
input rstn,
input start,
output reg [WIDTH-1:0] triangle_out

① → Clarify Question:

✓ Max amplitude?

✓ what does start mean?



for width = 8 $2^8 = 255$

0 → 1 → 2 → 3 → ... 255 → 254 → ... 0

② approach. I need up-down internal signal

up-down = 0

up-down = 1

triangle-out signal
is counter only

- ③ verification.
 - ✓ Reset check
 - ✓ MAX Edge detection

④ I implemented triangular wave generator using counter to direction bit.

direction bit is needed because once ~~at~~ MAX state reached, counter decrements

if $start = 1$ ✓ up down logic started
 $start = 0$ ✓ hold current value

Ques Can this be implemented as an FSM??
✓ yes

